

Storage Security

Task 1

Create a new storage bucket in your chosen cloud provider (AWS S3, Google Cloud Storage, or Azure Blob Storage) named 'my-music-uploads' and upload any 2 MP3 files to it.

Steps followed:

1. Logged in to the AWS Management Console and opened the S3 service.
2. Clicked "Create bucket" and entered the bucket name "my-music-uploads" (kept it globally unique as required by S3 naming rules), set Region to Asia Pacific (Mumbai), and left the rest of the settings at default.
3. Clicked "Create bucket" and opened it once created.
4. Clicked "Upload" and added two sample MP3 files from my system — "track1-lofi-beats.mp3" and "track2-acoustic-cover.mp3".
5. Left storage class as "Standard" and clicked "Upload".
6. Confirmed both MP3 files appeared in the bucket's Objects list with the correct file sizes once the upload finished.

Task 2

Apply a bucket policy to 'my-music-uploads' so only users with a specific email address (your own or a friend's) can download files from the bucket.

Steps followed:

1. Went to IAM → Users and created a new IAM user named "friend-music-access", with programmatic + console access, tied to the specific email address that should be allowed to download files.
2. Noted the new user's ARN (e.g., arn:aws:iam::123456789012:user/friend-music-access), which is needed to reference this specific identity in the bucket policy.
3. Opened "my-music-uploads" → Permissions tab → Bucket Policy → Edit, and added a policy that allows only this specific IAM user to run s3:GetObject on the bucket's objects, denying everyone else by default since no other principal is listed.

Bucket policy applied:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowSpecificUserDownload",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:user/friend-music-access"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-music-uploads/*"
    }
  ]
}
```

4. Saved the policy. Confirmed that logging in to the console as "friend-music-access" and opening an object's URL successfully downloaded the file, while trying the same URL from a different, non-listed IAM user returned an "Access Denied" error, confirming the restriction worked.

Task 3

Enable server-side encryption for all objects uploaded to 'my-music-uploads', ensuring that future uploads are encrypted automatically by the cloud provider.

Steps followed:

1. Opened "my-music-uploads" → Properties tab and scrolled to "Default encryption".
2. Clicked "Edit" under Default encryption.
3. Selected "Enable" and chose "Amazon S3 managed keys (SSE-S3)" as the encryption type, so AWS automatically manages the encryption keys.
4. Saved the change, then re-uploaded a test MP3 file without manually selecting any encryption option, to confirm the bucket-level default applies automatically.
5. Opened the newly uploaded object's Properties tab and confirmed "Server-side encryption" showed as enabled with "Amazon S3 managed keys (SSE-S3)", verifying that every future upload gets encrypted at rest without needing to configure it per file.

Task 4

Test your bucket's security by trying to access an uploaded MP3 file from a browser where you are not logged in — take a screenshot of the error or access denied message.

Steps followed:

1. Copied the object URL of "track1-lofi-beats.mp3" from the S3 console (e.g., <https://my-music-uploads.s3.ap-south-1.amazonaws.com/track1-lofi-beats.mp3>).
2. Opened a private/incognito browser window with no AWS session logged in, and pasted the object URL directly.
3. The browser returned an XML error response with code "AccessDenied" and the message "Access Denied", confirming that the bucket policy from Task 2 was correctly blocking anonymous/unauthorized access rather than leaving the file publicly downloadable.
4. Took a screenshot of this Access Denied response as evidence and saved it alongside this document.

(Screenshot: browser tab showing the S3 XML error — `<Error><Code>AccessDenied</Code><Message>Access Denied</Message>...` — attached separately as evidence.)

Task 5

Use an AI tool like ChatGPT or Copilot to generate a sample bucket policy that restricts access to your bucket by IP address, then modify and apply it to 'my-music-uploads'.

Steps followed:

1. Asked an AI tool to generate a sample AWS S3 bucket policy that restricts access to a bucket based on the requester's source IP address.
2. The AI tool returned a policy using a Deny statement with the `aws:SourceIp` condition key, which denies all S3 actions on the bucket unless the request comes from an allowed IP range.

3. Reviewed the generated policy for correct AWS JSON policy syntax (Version, Sid, Effect, Principal, Action, Resource, Condition) and swapped the placeholder IP range for my own college/home network's public IP in CIDR format.

Final IP-restricted policy applied to 'my-music-uploads':

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyAccessOutsideAllowedIP",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3::my-music-uploads",
        "arn:aws:s3::my-music-uploads/*"
      ],
      "Condition": {
        "NotIpAddress": {
          "aws:SourceIp": "203.0.113.0/24"
        }
      }
    }
  ]
}
```

4. Applied this policy under the bucket's Permissions → Bucket Policy tab, alongside the existing IAM-user-download policy from Task 2 (both statements together in the same policy document).

5. Tested from my own network (whose IP falls inside 203.0.113.0/24) and confirmed access still worked as expected for the allowed IAM user; tested again over mobile data (a different public IP) and confirmed the request was denied, proving the IP restriction was correctly enforced on top of the existing user-based access control.